Design System · Bibliotela

Especificação dos tokens e dos componentes da Bibliotela, o app de livros, séries e filmes. Foi extraída do protótipo do Figma Make (“Diário pessoal de entretenimento”) e organizada na estrutura do Design System da Carteira, para servir de referência ao pacote Flutter `bibliotela_design_system`, ao arquivo do Figma e às telas. Onde o protótipo tem um problema de acessibilidade ou diverge do contrato da API, o documento diz isso e propõe o ajuste (seção 8).

1. Princípios

- **Quente e editorial.** Fundos em off-white quente, uma única cor de destaque (azul-petróleo) e muito espaço. Parece uma estante, não um painel.
- **Capas são ícones.** O app não usa imagens: cada tipo (livro, série, filme) tem um ícone e uma cor.
- **A cor nunca fala sozinha.** Status e nota sempre têm texto (“Lendo”, “9 de 10”) além da cor e das estrelas.
- **Dois temas, os mesmos nomes.** Claro e escuro têm exatamente os mesmos tokens, e nenhum componente lê uma cor direto: tudo passa pelo tema.
- **Toque confortável.** Qualquer alvo de toque tem pelo menos 44 × 44.

2. Estrutura do pacote

Mesma organização do `carteira_design_system`: tokens, tema, componentes, testes e um app de galeria (`example/`).

```
bibliotela_design_system/
├── pubspec.yaml                 name: bibliotela_design_system
├── lib/
│   ├── bibliotela_design_system.dart     (arquivo único de exportação)
│   └── src/
│       ├── tokens/    app_colors · app_spacing · app_radius · app_typography · app_durations
│       ├── theme/     app_theme (claro/escuro) · app_semantic_colors
│       └── components/  um arquivo por componente (seção 5)
├── test/                        um teste por componente, tema e contraste
└── example/                     galeria: tokens, botões, campos, seletores, itens, estados
```

Quase tudo se reaproveita da Carteira. Troca-se a paleta e acrescentam-se as peças próprias de mídia:

Reaproveita da Carteira (muda só a cor)	Parente adaptado	Novo na Bibliotela
AppButton, AppTextField, PasswordField, AppScaffold, SectionHeader, StateView, AppAvatar, Gap	ItemTile (de TransactionTile), SummaryCard (de BalanceCard), GenreBar (de CategoryBar)	StarRating, StarRatingInput, StatusChip, MediaCover, AppSegmentedControl, AppFilterChip, AppDropdown, AppDateField, YearChip, DetailRow, NoteCard, ConfirmDialog, AppIconButton

Saem de cena: `MoneyText` (não há dinheiro) e os tokens `income`, `expense` e `transfer`.

3. Tokens

3.1 Cores

Duas paletas com os mesmos nomes de campo, como em `AppColorPalette`. Os valores abaixo já incluem os ajustes de contraste da seção 8.

Token	Claro	Escuro	Uso
`background`	`#f8f6f3`	`#16181d`	fundo das telas

Token	Claro	Escuro	Uso
`surface`	`#ffffff`	`#1f2228`	cartões, campos, barra de abas, diálogo
`border`	`#e3dfd8`	`#2c3038`	bordas e divisórias
`track`	`#ece9e4`	`#2c3038`	trilho das barras e do controle segmentado
`onSurface`	`#1f2328`	`#eceae6`	texto principal
`onSurfaceMuted`	`#6b7075`	`#9aa0a6`	texto secundário e ícones de campo
`primary`	`#0f6e8c`	`#7db8d0`	botão, aba ativa, links, FAB
`onPrimary`	`#ffffff`	`#0d2630`	texto e ícone sobre `primary`
`primaryContainer`	`#dceef4`	`#1f3a47`	aba ativa, chip selecionado, chip de ano
`error`	`#b5473e`	`#e0877e`	erro de campo, botão de excluir
`onError`	`#ffffff`	`#2a0f0b`	texto sobre `error`

Cores semânticas, no `AppSemanticColors` (extensão do tema, lida com `context.semanticColors`):

Token	Claro	Escuro	Uso
`want`	`#46627f`	`#8fb0d1`	status “quero” (texto e fundo suave)
`progress`	`#8a5a0f`	`#e0a94a`	status “em andamento”
`done`	`#25694b`	`#6fbf9a`	status “concluído”
`book`	`#0f6e8c`	`#7db8d0`	ícone e fundo suave de livro
`series`	`#7a5ba6`	`#b79ae0`	ícone e fundo suave de série
`movie`	`#c2603f`	`#c2603f`	ícone e fundo suave de filme
`starFilled`	`#a87300`	`#d9a21b`	estrela cheia
`starEmpty`	`#d5d1c9`	`#3a3e46`	estrela vazia (decorativa)
`muted`	`#6b7075`	`#9aa0a6`	atalho para `onSurfaceMuted` (igual à Carteira)

Fundos suaves. O fundo de um chip de status ou de uma capa é a própria cor com **14% de opacidade** sobre `surface`. Não existe token separado para isso; calcula-se com `Color.alphaBlend`.

3.2 Tipografia

Escala única, com os nomes de `TextTheme`. O protótipo usa Inter (400, 500, 600 e 700). No Flutter, a decisão é usar a **fonte do sistema**, como na Carteira, ou empacotar a Inter como asset (sem dependência extra). Os tamanhos valem nos dois casos.

Token	Tamanho / linha	Peso	Onde aparece
`displayLarge`	40 / 44, espaçamento −4%	600	número grande do resumo (“12”)
`headlineMedium`	28 / 34	600	nome do app no login
`titleLarge`	24 / 32	600	título da barra superior
`titleMedium`	22 / 28	600	título do detalhe, nome no perfil
`titleSmall`	15 / 20	600	título do item na lista
`bodyLarge`	15 / 22	400	observação, texto corrido
`bodyMedium`	14 / 20	400 e 500	dados do detalhe, busca; valor da linha em 500
`bodySmall`	13 / 18	400 e 500	rótulo de campo (500), erro, legenda do resumo

Token	Tamanho / linha	Peso	Onde aparece
`labelLarge`	15 / 20	600	texto de botão
`labelMedium`	12 / 17	400	legenda do item (gênero · dado do tipo), rótulo da aba
`labelSmall`	12 / 16, espaçamento +6%, CAIXA ALTA	600	título de seção (“POR GÊNERO”, mês, “NA FILA”)
`chip`	11 / 15	600	chip de status, data curta da lista (400)

Números com algarismos tabulares quando alinhados em coluna (`FontFeature.tabularFigures()`), como na Carteira. Formato brasileiro sempre: `3.480` páginas, `7,8` de média.

3.3 Espaçamento e dimensões

Escala de 4 pt, igual à da Carteira. Margem lateral das telas: `lg` (16).

Token	Valor	Token	Valor
`xxs`	2	`lg`	16 (margem da tela)
`xs`	4	`xl`	24
`sm`	8	`xxl`	32
`md`	12	`xxxl`	48

Dimensões fixas do protótipo:

Elemento	Medida	Elemento	Medida
Barra superior	64 de altura	Botão e campo	mínimo 48
Barra de abas	72 de altura	Controle segmentado	mínimo 46
Botão de ícone	44 × 44	Chip de filtro	38 de altura
FAB	56 × 56, direita 18, base 88	Chip de status	mínimo 28 de altura
Capa pequena	48 × 64	Item da lista	mínimo 88 de altura
Capa grande	96 × 128	Avatar do perfil	96
Ícone do app	64 × 64	Chip de ano	34 de altura

3.4 Raios, sombras e movimento

Raio	Valor	Onde
`sm`	8	capa pequena
`md`	12	botões, campos, controle segmentado, capa grande, nota, botão de ícone (o botão interno do segmentado usa 9)
`lg`	16	cartões, lista, FAB, diálogo
`xl`	18	ícone do app no login
`pill`	999	chips, chip de ano, barras, aba ativa, avatar

Sombra	Valor	Onde
`fab`	0 8 22, `primary` a 32%	botão flutuante
`appIcon`	0 8 20, `primary` a 26%	ícone do app
`menu`	0 10 30, preto a 12%	menu do chip de ano

Sombra	Valor	Onde
`dialog`	0 20 50, preto a 33%	diálogo de confirmação

Movimento (`AppDurations`): `fast` 150 ms (pressionar o botão: escala 0,985), `normal` 220 ms, `slow` 400 ms. O indicador de carregamento do botão gira em 700 ms, linear.

3.5 Ícones

Traço de 1,75, grade 24, pontas arredondadas, sem preenchimento. No Flutter, os equivalentes de `Icons` (versão “outlined”):

Uso	Flutter	Uso	Flutter
Livro, aba Biblioteca	`menu_book_outlined`	Voltar	`chevron_left`
Filme	`movie_outlined`	Editar	`edit_outlined`
Série	`tv_outlined`	Excluir	`delete_outline`
Perfil	`person_outline`	E-mail	`mail_outline`
Tema claro	`light_mode_outlined`	Senha	`lock_outline`
Tema escuro	`dark_mode_outlined`	Mostrar senha	`visibility_outlined`
Buscar	`search`	Sair	`logout`
Novo item	`add`	Foto	`image_outlined`
Abrir lista (ano, gênero)	`expand_more`	Ícone do app	`personalizado (livro com play)`

4. Tema

- `AppTheme.light()` e `AppTheme.dark()` montam o `ThemeData` (Material 3) a partir de `AppColors.light` e `AppColors.dark`, como na Carteira.
- `AppSemanticColors` ganha `want`, `progress`, `done`, `book`, `series`, `movie`, `starFilled`, `starEmpty` e `muted`, e perde `income`, `expense`, `transfer` e `warning`.
- Temas de componente a configurar: `appBarTheme`, `cardTheme`, `inputDecorationTheme`, `filledButtonTheme`, `navigationBarTheme` (indicador 64 × 30 em `primaryContainer`), `floatingActionButtonTheme` (56, raio 16) e `dialogTheme` (raio 16).
- O controle segmentado do protótipo (fundo em trilho, aba ativa em `primaryContainer`) **não** se obtém só com o tema do `SegmentedButton` do Material; por isso ele é um componente próprio.
- Troca de tema em tempo de execução (ícone sol/lua): o app guarda um `ThemeMode` e passa para o `MaterialApp`.

5. Componentes

Componente	Origem	Onde aparece
AppButton (primário, secundário, texto, perigo)	Carteira (+ perigo)	login, formulário, perfil, diálogo
AppTextField, PasswordField	Carteira (rótulo acima)	login, busca, formulário, perfil
AppDropdown, AppDateField	novo	formulário
AppSegmentedControl	novo	filtro de status, tipo e status do formulário
AppFilterChip, YearChip	novo	filtro de tipo, resumo do ano
StatusChip	novo	lista, detalhe
StarRating, StarRatingInput	novo	lista, resumo, detalhe, formulário
MediaCover	novo	lista, detalhe
ItemTile	de TransactionTile	lista da biblioteca
SummaryCard	de BalanceCard	topo da biblioteca
GenreBar	de CategoryBar	seção “Por gênero”
SectionHeader	Carteira	gênero, mês, “Na fila”, dados, nota, observação
AppScaffold, AppIconButton	Carteira (+ botão de ícone)	todas as telas
AppBottomTabs (NavigationBar), AppFab	tema da Carteira	biblioteca e perfil
ConfirmDialog	novo	excluir item
StateView	Carteira (+ ação no vazio)	carregando, vazio, erro
DetailRow, NoteCard	novo	detalhe
AppAvatar, Gap	Carteira	perfil, espaçamento

`AppButton`

Igual ao da Carteira, com a variante `danger`. Altura mínima 48, raio 12, texto `labelLarge`. Quando `isLoading`, mostra um indicador de 18 no lugar do ícone e fica desabilitado.

```
AppButton({required String label, required VoidCallback? onPressed,
           AppButtonVariant variant = primary, bool isLoading = false,
           IconData? icon, bool expanded = true})
AppButton.secondary(...)   // borda `border`, fundo `surface`
AppButton.text(...)        // `primary`, sem fundo, expanded = false
AppButton.danger(...)      // fundo `error`, texto `onError`, expanded = false
```

- **Estados:** normal, pressionado (escala 0,985), desabilitado (opacidade 70%) e carregando.
- O botão destrutivo só aparece dentro de diálogo de confirmação, nunca solto na tela.

`AppTextField e PasswordField`

Mesmas propriedades da Carteira. A diferença visual: no protótipo, o **rótulo fica acima** do campo (13/18, peso 500, 6 de distância), e não flutuando dentro da borda. A caixa tem altura mínima 48, raio 12, borda 1, fundo `surface` e ícone de 19 em `onSurfaceMuted`.

```
AppTextField({required String label, TextEditingController? controller,
              String? hint, String? errorText, String? helperText,
              TextInputType? keyboardType, TextInputAction? textInputAction,
              TextCapitalization textCapitalization, int maxLines = 1,
              IconData? prefixIcon, Widget? suffix,
```

```
            ValueChanged<String>? onChanged, ValueChanged<String>? onSubmitted})
PasswordField({String label = 'Senha', ...})   // botão de mostrar/ocultar (44 × 44)
```

- **Foco:** borda `primary` e anel de 3 em `primary` a 15%. **Erro:** borda `error` e mensagem 13/18 em `error` logo abaixo.
- Campo de várias linhas (observação): 3 linhas, sem redimensionar.
- A busca da biblioteca é um `AppTextField` com `prefixIcon: Icons.search` e dica “Buscar por título ou autor”.

AppDropdown e AppDateField

Mesma aparência do campo de texto, com `expand_more` à direita (dropdown) ou o calendário (data). Os dois abrem o seletor nativo.

```
AppDropdown<T>({required String label, required T value,
                required List<AppDropdownItem<T>> items,
                required ValueChanged<T> onChanged, String? errorText})

AppDateField({required String label, required DateTime? value,
              required ValueChanged<DateTime?> onChanged, String? errorText,
              DateTime? firstDate, DateTime? lastDate})
```

- O gênero tem a opção “Sem gênero”, que vira `genre: null` na API.
- A data aparece como `dd/mm/aaaa` e vai para a API só como `YYYY-MM-DD`, sem hora nem fuso.

AppSegmentedControl

Uma fileira de opções sobre um trilho. Serve para o filtro de status (“Todos / Quero / Em andamento / Concluídos”), para o tipo e para o status do formulário.

```
AppSegmentedControl<T>({required List<AppSegment<T>> options,
                        required T value, required ValueChanged<T> onChanged})
AppSegment<T>({required T value, required String label})
```

- Trilho: fundo `track`, padding 3, espaço de 3 entre opções, raio 12, altura mínima 46.
- Opção: raio 9, texto 12/15 (11 em telas de até 390). Ativa: fundo `primaryContainer`, texto `primary`, peso 600.
- No formulário, os rótulos de status mudam com o tipo: Quero ler / Lendo / Lido (livro) e Quero assistir / Assistindo / Assistido (série e filme).

AppFilterChip e YearChip

```
AppFilterChip({required String label, required bool selected, required VoidCallback onTap})
YearChip({required int year, required List<int> years, required ValueChanged<int> onChanged})
```

- Chip de filtro: altura 38, pílula, padding 15, texto 13. Selecionado: fundo `primaryContainer`, sem borda, texto `primary` 600. Não selecionado: borda e fundo `surface`.
- Os chips ficam numa linha que rola na horizontal, com 8 de espaço, e alcançam a borda da tela (margem negativa de 16).
- Chip de ano: altura 34, fundo `primaryContainer`, texto `primary` 600 e seta de 16. Abre um menu de 92 de largura, raio 12, com itens de 38 de altura.

StatusChip

```
StatusChip({required MediaStatus status, required MediaType type})
enum MediaStatus { want, inProgress, done }     // API: want, in_progress, done
enum MediaType { book, series, movie }
```

Status	Livro	Série e filme	Cor do texto	Fundo
`want`	Quero ler	Quero assistir	want	want a 14%

Status	Livro	Série e filme	Cor do texto	Fundo
`inProgress`	Lendo	Assistindo	progress	progress a 14%
`done`	Lido	Assistido	done	done a 14%

- Altura mínima 28, pílula, padding 9, texto `chip` (11/15, 600), sem quebra de linha.

`StarRating e StarRatingInput`

A nota é um **inteiro de 1 a 10**; as estrelas mostram **nota ÷ 2**, com meia estrela. 8 = 4 estrelas, 9 = 4,5 estrelas.

```
StarRating({required int? rating, StarSize size = StarSize.small})   // 14 | 20 | 28
StarRatingInput({required int? value, required ValueChanged<int?> onChanged})
String ratingLabel(int rating)   // '9 de 10 · 4,5 estrelas'  ·  '2 de 10 · 1 estrela'  ·  '1 de 10 · meia estre
```

- Tamanhos 14 (lista e resumo), 20 e 28 (detalhe e formulário; com espaço de 2). Cada estrela é a vazia com a cheia recortada por cima (0%, 50% ou 100%).
- **Entrada:** cada estrela tem alvo de pelo menos 36 × 44. Tocar na **metade esquerda** dá meia estrela; na direita, estrela inteira. “Remover nota” aparece quando há nota.
- **Acessibilidade:** o rótulo falado é “Nota 9 de 10”. Na lista, as estrelas aparecem sem texto, então o rótulo é obrigatório.
- A nota só existe com status concluído. Ao sair desse status, a nota é descartada.

`MediaCover`

```
MediaCover({required MediaType type, MediaCoverSize size = MediaCoverSize.small})   // 48×64 | 96×128
```

- Fundo: a cor do tipo (`book`, `series`, `movie`) a 14% sobre `surface`. Ícone na cor do tipo: 24 (pequena) ou 36 (grande).
- Raio 8 (pequena) e 12 (grande). Livro: `menu_book_outlined`. Série: `tv_outlined`. Filme: `movie_outlined`.

`ItemTile`

Linha da lista. Substitui o `TransactionTile`: no lugar do valor colorido, entram estrelas ou o chip de status.

```
ItemTile({required MediaType type, required String title, required String subtitle,
          required MediaStatus status, int? rating, String? dateLabel, VoidCallback? onTap})
```

- Altura mínima 88, padding vertical 11, espaço de 12. Esquerda: `MediaCover` pequena. Centro: título `titleSmall` em uma linha com reticências e, abaixo, o `subtitle` em `labelMedium` `muted`, até 2 linhas.
- `subtitle` por tipo: livro “Ficção científica · Frank Herbert”; série “Suspense · 2 temporadas”; filme “Drama · 3 h 01 min”.
- **Direita:** concluído com nota mostra `StarRating` pequena e, embaixo, a data curta (“06 set”, texto `chip` 400 `muted`). Qualquer outro caso (quero, em andamento ou concluído sem nota) mostra o `StatusChip`.
- A lista fica num contêiner `surface` com borda, raio 16 e padding horizontal 12; entre itens, uma divisória de 1 em `border` (sem divisória no último).

`SummaryCard`

Cartão do topo da biblioteca. Substitui o `BalanceCard`.

```
SummaryCard({required int year, required List<int> years, required ValueChanged<int> onYearChanged,
             required int booksFinished, required int pagesRead,
             required int movies, required int series, double? averageRating})
```

- Cartão `surface`, borda, raio 16, padding 16. Linha de cima: “Resumo do ano” (13, `muted`) e o `YearChip`.
- Número grande: `displayLarge` com a unidade (“livros lidos”) em 20 logo ao lado. Abaixo, “3.480 páginas lidas” em `primary`, 14/500.
- Três métricas lado a lado com divisórias (proporção 1 : 1 : 1,35): filmes, séries e “Nota média” com `StarRating` pequena e o valor (“7,8”). Separadas do bloco de cima por uma linha de 1.

- As estrelas da média mostram a média arredondada ÷ 2 (7,8 arredonda para 8, que dá 4 estrelas). Sem nenhuma nota no ano, mostra “—”.
- Números sempre no formato brasileiro (`intl`, `pt_BR`).

GenreBar

```
GenreBar({required String label, required int count, required double share})   // share: 0 a 1
```

- Linha: nome (13) à esquerda; “14 · 33%” (13, `muted`) à direita. Embaixo, a barra: altura 6, pílula, preenchimento `primary` sobre `track`. 11 de espaço entre linhas.
- Mostram-se os 5 maiores gêneros; itens sem gênero aparecem como “Sem gênero”.

SectionHeader

- Igual ao da Carteira: `labelSmall` (12/16, 600, CAIXA ALTA, +6%) em `muted`, 10 de espaço abaixo. Serve de título de mês (“SETEMBRO DE 2026”), de “NA FILA”, “POR GÊNERO”, “DADOS”, “NOTA” e “OBSERVAÇÃO”.
- Helper `monthHeading(DateTime)` devolve o texto do mês, em maiúsculas.

AppScaffold, AppIconButton, AppBottomTabs e AppFab

- **Barra superior:** altura 64, padding 10 × 16, título `titleLarge`, divisória de 1 embaixo. Voltar à esquerda; ações à direita. `AppScaffold(title, actions, leading, showAppBar, padded, bottom, floatingActionButton)`, como na Carteira.
- **AppIconButton:** 44 × 44, raio 12; pressionado, fundo `primaryContainer`. Variante de perigo (lixeira) em `error`. Sempre com `tooltip`.
- **Barra de abas:** `NavigationBar` com 2 destinos (Biblioteca, Perfil), altura 72, fundo `surface`, borda no topo, indicador 64 × 30 em `primaryContainer`. Ativa: `primary` e peso 600. As duas abas ficam vivas (`IndexedStack`).
- **FAB:** 56 × 56, raio 16, `primary`, ícone `add` de 28 e a sombra `fab`. Fica a 16 acima da barra de abas, à direita (18 da borda).

ConfirmDialog

```
Future<bool> showConfirmDialog(BuildContext context, {required String title,
    required String message, String confirmLabel = 'Confirmar',
    String cancelLabel = 'Cancelar', bool destructive = false})
```

- Fundo escuro a 53%, cartão `surface` com raio 16, padding 22 e a sombra `dialog`. Título 19, mensagem 14 em `muted`, botões alinhados à direita com 6 de espaço.
- Excluir item: título “Excluir item?”, mensagem “Essa ação não pode ser desfeita.”, botões “Cancelar” (texto) e “Excluir” (perigo).

StateView

```
StateView.loading({String? message})
StateView.empty({required String message, String? description, IconData icon, String? actionLabel, VoidCallback?
StateView.error({required String message, VoidCallback? onAction, String actionLabel = 'Tentar novamente'})
```

Estado	Texto	Ação
Vazio, sem filtro	“Sua biblioteca está vazia” e “Toque em + para adicionar o primeiro item.”	—
Vazio, com busca ou filtro	“Nenhum item encontrado” e “Tente ajustar a busca ou os filtros.”	“Limpar filtros”
Erro de carregamento	“Não foi possível carregar sua biblioteca.”	“Tentar novamente”

Estado	Texto	Ação
Carregando	indicador circular; no resumo e na lista, esqueletos (blocos em `track`)	—

`DetailRow, NoteCard, AppAvatar e Gap`

```
DetailRow({required String label, required String value, bool italic = false})
NoteCard({String? text})        // sem texto: 'Sem observação', itálico, `muted`
```

- **DetailRow:** altura mínima 46, divisória de 1 embaixo. Rótulo 14 `muted` ocupando 42%; valor 14, peso 500, alinhado à direita. Valor vazio aparece como “—”.
- O detalhe usa 5 linhas: Tipo, Gênero, Início, Fim e (Páginas, Temporadas ou Duração, conforme o tipo). Datas como `dd/mm/aaaa`; duração como “181 minutos”.
- **NoteCard:** raio 12, borda, fundo `surface`, padding 14, texto `bodyLarge`.
- **AppAvatar** (perfil: 96, fundo `primaryContainer`, iniciais em `primary`, 28/600) e **Gap** são os da Carteira.

6. Quais peças cada tela usa

Tela	Componentes
Login	AppScaffold (sem barra), ícone do app, AppTextField, PasswordField, AppButton, mensagem de erro
Biblioteca	AppScaffold (título e botão de tema), SummaryCard, SectionHeader, GenreBar, AppTextField (busca), AppSegmentedControl (status), AppFilterChip (tipo), SectionHeader (mês), ItemTile, StateView, AppFab, AppBottomTabs
Detalhe	AppScaffold (voltar, editar, excluir), MediaCover (grande), StatusChip, StarRating (28), DetailRow, SectionHeader, NoteCard, ConfirmDialog
Novo e editar item	AppScaffold (voltar), AppSegmentedControl (tipo e status), AppTextField, AppDropdown, AppDateField, StarRatingInput, AppButton (fixo no rodapé)
Perfil	AppScaffold, AppAvatar, SectionHeader, AppTextField (nome e foto), AppButton, AppButton.secondary (sair), AppBottomTabs

Estados que toda tela com dados precisa ter: carregando, vazio, erro com “Tentar novamente” e, nos formulários, erro por campo e botão em carregamento. O protótipo só mostra o vazio; os outros estão definidos aqui (StateView).

7. Galeria e testes

Como na Carteira, o pacote traz um app de exemplo (`example/`) e testes de componente.

Página da galeria	Mostra
Tokens	paleta nos dois temas, escala de texto, espaçamentos, raios e sombras
Botões	primário, secundário, texto e perigo; normal, desabilitado e carregando
Campos	texto, senha, busca, várias linhas, dropdown, data; normal, foco e erro
Seletores	controle segmentado, chips de filtro, chip de ano, StarRating (3 tamanhos) e StarRatingInput
Itens	ItemTile (os cinco casos), SummaryCard, GenreBar, DetailRow, NoteCard
Estados	StateView (carregando, vazio, erro), ConfirmDialog, StatusChip em todos os status

- **Um teste por componente** (renderiza, estados, rótulos acessíveis) e um teste do tema que garante que `light()` e `dark()` têm os mesmos campos.

- **Teste de contraste:** calcula a razão entre os pares da seção 8 e falha se algum cair abaixo do mínimo. É ele que impede uma troca de cor de quebrar a acessibilidade sem ninguém notar.

8. Ajustes em relação ao protótipo

Medi o contraste (WCAG 2.1) de todos os pares do protótipo. **Texto principal, texto secundário, links e botões já passam** nos dois temas. O que não passa foi ajustado neste documento. Mínimos: 4,5 para texto pequeno (o chip tem 11 px) e 3,0 para ícones e estrelas.

8.1 Pares que já passam

Par	Claro	Escuro
texto principal sobre o fundo	14.65	14.78
texto secundário sobre o fundo	4.64	6.73
texto secundário sobre a superfície	5.00	6.04
`primary` sobre o fundo (links, valores)	5.36	8.15
`primary` sobre `primaryContainer` (aba, chip de ano)	4.84	5.49
`onPrimary` sobre `primary` (botão)	5.79	7.21
`error` sobre a superfície (mensagem de erro)	5.34	6.02

8.2 O que foi ajustado

Item	Tema	Protótipo	Contraste	Proposta	Contraste	Mínimo
chip “quero”	claro	`#5b7a99`	3.77	`#46627f`	5.19	4.5
chip “em andamento”	claro	`#b7791f`	3.11	`#8a5a0f`	4.86	4.5
chip “concluído”	claro	`#2f7d5b`	4.16	`#25694b`	5.34	4.5
chip “quero”	escuro	`#5b7a99`	3.06	`#8fb0d1`	5.40	4.5
ícone de livro	escuro	`#0f6e8c`	2.45	`#7db8d0`	5.58	3.0
ícone de série	escuro	`#7a5ba6`	2.58	`#b79ae0`	5.13	3.0
estrela cheia	claro	`#d9a21b`	2.30	`#a87300`	4.10	3.0

Fundo do chip e da capa: a própria cor a 14% sobre a superfície. A estrela vazia (1,5) é decorativa; a nota sempre vem também em texto ou no rótulo falado.

8.3 Botão de perigo

O protótipo usa texto branco no botão de excluir nos dois temas. No escuro, branco sobre `error` (`#e0877e`) dá 2.65, abaixo de 4,5. A paleta define `onError` para isso: claro `#ffffff` (5.34) e escuro `#2a0f0b` (6.77).

8.4 Outras diferenças do protótipo

- **Status “progress”.** O protótipo escreve `progress`; o contrato da API escreve `in_progress`. O Flutter usa `MediaStatus.inProgress` e converte na borda (`fromJson`/`toJson`).
- **Vírgula decimal.** O protótipo mostra “4.5 estrelas”; o correto no Brasil é “4,5”. Vale `ratingLabel`, e “meia estrela” para a nota 1.
- **Ordem dos meses.** O protótipo ordena os grupos de mês pelo nome (ordem alfabética), o que erra em casos como julho e junho. No app, ordenar pela data, do mais recente para o mais antigo, com “Na fila” sempre no fim, como o contrato define.
- **Estrelas da média.** O protótipo fixa 4 estrelas na nota média. O componente deve derivá-las da média (média arredondada ÷ 2).

- **Cores de tipo e “quero” no tema escuro.** O protótipo não as muda entre os temas. Aqui, o escuro tem versões mais claras para “quero”, livro e série. O filme já passava (3,51 no claro e 3,28 no escuro) e ficou como estava.
- **Rótulo do campo.** No protótipo, o rótulo fica acima do campo; na Carteira, flutuava dentro da borda. A Bibliotela segue o protótipo.
- **Estados.** O protótipo não tem carregando nem erro, nem “Limpar filtros”; estão definidos no StateView.
- **Fonte.** O protótipo usa Inter. A decisão entre a fonte do sistema e empacotar a Inter fica para o pacote Flutter (seção 3.2).

9. Convenções para o arquivo do Figma

- **Páginas:** “Tokens”, “Componentes” e “Telas”.
- **Variáveis de cor,** uma coleção “Cor” com os modos “Claro” e “Escuro”. Nomes em barras, iguais aos do código: `color/background`, `color/surface`, `color/primary`, `color/primaryContainer`, `status/want`, `status/progress`, `status/done`, `type/book`, `type/series`, `type/movie`, `star/filled`, `star/empty`.
- **Variáveis numéricas:** `space/xs` ... `space/xxxl` e `radius/sm`, `radius/md`, `radius/lg`, `radius/pill`.
- **Estilos de texto** com os nomes da escala (`display/large`, `title/large`, `body/medium`, `label/small`...).
- **Componentes** com nomes do pacote e propriedades de variante: `Button` (variante, estado, com ícone), `StatusChip` (status, tipo), `StarRating` (tamanho, nota), `MediaCover` (tipo, tamanho), `ItemTile` (caso: com nota, só chip), `SegmentedControl`, `Field` (estado) e os demais da seção 5.
- **Auto layout** em todo componente, com as medidas da seção 3.3, e cada componente mostrado nos dois temas lado a lado.